Studio Nx

Backend(FastAPI/Python) 15 — hash_password()

2026-10-07

Backend(FastAPI/Python) 15 — `hash_password()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-04 회원가입 — 비밀번호 해시 저장 · PRD 01 FR-102
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	14 check_password_rules() · 16 verify_password()

1. 하는 일

비밀번호를 **되돌릴 수 없는 글자(해시)** 로 바꾼다. DB 에는 비밀번호 대신 이 해시만 저장한다.

`hash_password("Abcdef1!")` → `"$2b$12$Q9xK…(60자)"`

해시의 부분	예	뜻
`$2b$`		bcrypt 방식
`12$`		비용(cost) 12 — 계산을 2^{12} 번 반복. 클수록 느림
다음 22자	`Q9xKc3…`	소금(salt) — 해시마다 새로 만든 무작위 값
나머지 31자		비밀번호 + 소금으로 계산한 결과

2. 언제 쓰나

쓰는 곳	하는 일
P1-04 `signup()` (21)	규칙 검사(14)를 통과한 비밀번호를 해시해 `users.password_hash` 에 저장
P1-09 `change_password()`	새 비밀번호를 해시해 바꿔 저장

로그인(P1-05)할 때는 이 해시를 되돌리지 않는다. 입력한 비밀번호를 같은 소금으로 다시 계산해 결과가 같은지 비교한다 (다음 16 `verify_password()`).

왜 비밀번호를 그대로 저장하지 않나

저장 방식	DB 가 유출되면
비밀번호 그대로	모든 회원의 비밀번호가 바로 드러남. 같은 비밀번호를 쓰는 다른 사이트까지 위험
단순 해시 (SHA-256 등)	계산이 빨라서 흔한 비밀번호를 초당 수십억 개 대입해 찾아냄
bcrypt (소금 + 느린 계산)	한 번 계산에 약 0.2초. 회원마다 소금이 달라 미리 만든 표로 찾을 수 없음

3. 코드

import (파일 맨 위)

```
import re
import string
```

```
import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다. **`import bcrypt` 를 이번에 추가했고**, 이 함수는 그것만 쓴다. `bcrypt` 패키지는 `requirements.txt` 에 이미 있다 (설치된 버전 5.0.0).

함수

```
def hash_password(plain: str) -> str:
    return bcrypt.hashpw(plain.encode("utf-8"), bcrypt.gensalt()).decode("ascii")
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와 `모듈.이름` 으로 씀 (예: `bcrypt.hashpw`)

한 줄을 안쪽부터 풀면 다음과 같다.

순서	식	값
①	`plain.encode("utf-8")`	`"Abcdef1!"` → `b"Abcdef1!"` (글자 → 바이트. bcrypt 는 바이트만 받음)
②	`bcrypt.gensalt()`	`b"$2b$12$Q9xKc3…"` — 새 소금 (기본 비용 12)
③	`bcrypt.hashpw(①, ②)`	`b"$2b$12$Q9xKc3…(60바이트)"` — 해시 (바이트)
④	`.decode("ascii")`	바이트 → 글자. DB 의 `VARCHAR` 칸에 넣으려고

식	뜻
`b"…"`	바이트 값. 글자(`str`)와 다른 타입
`"ascii"` 로 되돌림	bcrypt 해시는 영문 · 숫자 · `$./` 만 쓰므로 ASCII 로 충분

4. 흐름

1. 비밀번호를 바이트로 바꾼다.
2. 새 소금을 만든다.
3. 비밀번호와 소금으로 해시를 계산한다 (약 0.2초).
4. 글자로 바꿔 돌려준다.

5. 왜 이렇게

선택	이유
bcrypt	PRD FR-102 가 정한 방식. 오래 검증된 비밀번호 전용 해시
소금을 매번 새로	같은 비밀번호라도 회원마다 해시가 다르다. 한 사람의 비밀번호를 알아내도 같은 비밀번호를 쓰는 다른 회원이 드러나지 않는다
소금을 따로 저장하지 않음	해시 안(`$2b$12$` 뒤 22자)에 들어 있다. 칸 하나(`password_hash`)로 충분하다

선택	이유
비용 12 (기본값)	이 컴퓨터에서 한 번에 약 0.19초. 가입 · 로그인 한 번에는 느끼지 못하지만, 대입 공격에는 매우 느리다
72바이트 제한은 따로 처리하지 않음	bcrypt 5 는 72바이트가 넘으면 오류를 낸다. 비밀번호는 14 에서 최대 20자 · 영문 · 숫자 · 기호(1글자 = 1바이트)로 검사한 뒤 들어오므로 넘을 수 없다

6. 시험 (`tests/test_password.py` 에 5개 추가 · 모두 21개)

시험	확인한 것
모양	글자이고, `$2b$12$` 로 시작하고, 60자 (`VARCHAR(255)` 에 충분)
매번 다름	같은 비밀번호를 두 번 해시하면 결과가 다름 (소금)
원래 비밀번호만 맞음	`bcrypt.checkpw` 로 `"Abcdef1!"` 은 맞고 `"Abcdef1?"` 는 틀림
비밀번호가 보이지 않음	해시 안에 원래 글자가 없음
가장 긴 비밀번호 (20자)	오류 없이 해시되고 비교도 맞음

비교 시험은 다음 함수(16 `verify_password`) 대신 `bcrypt.checkpw` 를 직접 썼다.

전체 시험 97개 통과, `ruff` 검사 통과. 해시 한 번 약 190ms.